Revised Benchmarking Design Plan: Echo Server Track (v4)

This document serves as the single source of truth strictly for the Echo Server benchmark track (`select`, `poll`, `epoll`, `io_uring`). It supersedes Draft v3 by formally integrating the custom C++17 load generator (`echo_loadgen.cpp`), the hybrid open-loop pacing model, the continuous phase protocol, and extended correctness gates.

1. Executive Summary & Deliverables

We are benchmarking an echo server that waits for network events in four ways to empirically evaluate their scalability and latency under identical workloads.

Deliverables for the Echo Server Owner:

- `apps/echo.c` (App logic)
- `io/io_select.c`, `io_poll.c`, `io_epoll.c`, `io_uring.c` (I/O modules)
- **Fair Benchmarking Design Plan: Echo Server (Revised)**

Updated November 2026. This document supersedes the original Draft v3 strictly for the Echo Server track, incorporating the custom C++17 load generator and continuous-phase execution model.

1. Core Modifications from Original Draft v3

This revised plan explicitly modifies the following original directives:

- **Load Generator Replacement:** Replaces `tcpkali` with a custom C++17 client (`tools/echo_loadgen.cpp`).
- **Continuous Phase Execution:** Abandons the two-invocation method (warm-up process followed by measured process). The new load generator maintains persistent connections across a unified timeline (10s warm-up, 30s steady-state, 10s cool-down).
- **Deterministic Connection Interleaving:** Replaces the dual-`tcpkali` approach for E2 with single-process deterministic handshake interleaving, scattering active and idle connections uniformly across the server's file descriptor table.
- **Pipelined Open-Loop Pacing:** Modifies the strict lockstep rule (one message in-flight) to an open-loop `timerfd` architecture. The client permits bounded pipelining (up to 64 frames in-flight) to prevent Coordinated Omission during transient latency spikes, though the aggregate rate R must still be tuned so the average in-flight frames remain <1 during normal operation.

2. Code Structure & Plug-in Contract

The echo application logic resides entirely in `apps/echo.c`, interfacing with the four mechanisms (`io_select.c`, `io_poll.c`, `io_epoll.c`, `io_uring.c`) via the standard `io.h` contract.

- **Callbacks:** `on_read_done`, `on_write_done`, `on_accept`, and `on_close` are invoked exclusively from inside `io_run`, never recursively from an `io_` call.
- **Carry Slot:** Unframed trailing bytes must be stored in a per-connection carry slot allocated from a pool.

3. Sockets, Memory, and Framing Rules

- **No Delay:** `TCP_NODELAY` and `O_NONBLOCK` are globally enforced on all accepted sockets.

- **Memory Footprint:** `select`, `poll`, and `epoll` baseline modules must use a single shared 4 KB scratch buffer for reads. Only the `io_uring` baseline allocates a dedicated 4 KB buffer per connection.
- **Message Framing:** The echo server echoes only complete fixed-size frames (default 512 B). If multiple frames arrive in a single read, all complete frames must be grouped and returned in a single `io_write()`.

4. Load Generation & Measurement (`echo_loadgen.cpp`)

The custom load generator must provide:

- **Header & Telemetry:** A 32-byte binary header with a monotonic timestamp (`CLOCK_MONOTONIC`) and sequence number to calculate hardware-accurate RTT per frame.
- **Active/Idle Partitioning:** For Experiment E2, the client opens *C* total connections and interleaves the handshakes. Only the active fraction *A* is scheduled for transmission.
- **Client Health:** The load generator must utilize multi-threaded worker pools pinned to isolated client cores to ensure client CPU remains under 80%.

5. Benchmark Matrix & Feature Sweeps

- **E1 (Connection Scaling):** Total connections at 10, 100, 500, 1000, 5000, 10000 (`select` stops at 1000).
- **E1s (Saturation):** Unthrottled throughput and CPU per message at the E1 connection counts.
- **E2 (Active Ratio):** 1%, 10%, 50%, 100% active share of 1,000 and 10,000 total connections.
- **E3 (Load vs Latency):** 1,000 connections; rate steps from 10% up to saturation.
- **E4 (Message Size):** 100 connections; 64 B, 512 B, 4 KB, 16 KB frames.
- **Tier 1 Flags:** `--epoll-et`, `--accept-batch`, `--uring-multishot-accept`, `--uring-provided-buffers`, `--uring-multishot-recv`, `--uring-fixed-files`, `--uring-single-issuer`, `--uring-defer-taskrun`.

6. Execution Protocol

- **7 Repeats:** Every configuration cell is run 7 times; report the median and interquartile range.
- **Randomized Blocks:** Iterate through mechanisms in a shuffled block to distribute thermal throttling evenly.
- **Continuous Snapshots:** Send `SIGUSR1` to trigger JSON stats dumps and record `cpu.stat` exactly at the 10-second and 40-second marks of the single client process.
- **CPU Cost Calculation:** Defined as Δ CPU (user + system) + Δ NIC softirq divided by the total verified echoed messages during the 30-second window.

7. Correctness Gate

The echo server builds must pass the following `echo_gate.py` tests before benchmark results are accepted:

- **T1-T6:** Split messages, coalesced messages, byte-at-a-time, backpressure holds, and mid-message client resets/closes.
- **T7-T11, T14:** 10k connect-close leak checks, 1,000 concurrent limits, random chunking, half-closes, and slow-reader bounds.
- **T12 (Re-Enforced):** ASan and UBSan sanitized builds must run T1 to T11 cleanly to catch carry-slot and async lifetime violations.